pontIA

MÁSTER INGENIERÍA DE DATOS

Consultas 3

Operaciones DDL - Recursos

Daniel Trabas

pontIA

Índice

Objetivo 3

Operaciones DDL 3

Anexos 10

pontIA

Objetivo

El Lenguaje de Definición de Datos (DDL, Data Definition Language) constituye la base de la gestión de esquemas en cualquier sistema de gestión de bases de datos relacionales (RDBMS). En PostgreSQL, las operaciones DDL permiten crear y modificar la estructura física y lógica de los objetos que albergan los datos, de forma conceptualmente independiente a la manipulación de los registros contenidos en las tablas (DML[1]). Según la documentación oficial de PostgreSQL, esta división lógica es fundamental:

"Este bloque describe el uso del lenguaje SQL en PostgreSQL. Empezamos describiendo la sintaxis general de SQL, luego cómo crear tablas, cómo poblar la base de datos y cómo consultarla" (PostgreSQL 18.4 Documentation, Part II: The SQL Language).

[1] Data Manipulation Language: donde se encajan los comandos principales SELECT, INSERT, UPDATE y DELETE.

Operaciones DDL

- **CREATE:** define estructuras lógicas nuevas, como tablas, vistas, índices, esquemas o tipos de datos personalizados
- **ALTER:** modifica la estructura de un objeto existente sin necesidad de destruir el objeto ni perder sus datos históricos.
- **TRUNCATE:** vacía de manera eficiente y segura todo el contenido de una o más tablas, liberando el espacio en disco de forma inmediata.
- **DROP:** elimina de forma definitiva un objeto del catálogo del clúster (con opciones de borrado en cascada).
- **COMMENT:** documenta y asocia explicaciones técnicas legibles por humanos directamente en el catálogo de PostgreSQL.

CREATE TABLE y CREATE VIEW

Concepto y Aplicación de Vistas (CREATE VIEW)

Una vista en PostgreSQL es una 'tabla virtual' que no almacena físicamente datos por sí misma, sino que representa una consulta guardada en el catálogo del sistema. La consulta base se ejecuta de manera dinámica cada vez que la vista es referenciada en un SELECT.

Se puede crear una vista sobre la consulta, lo que hace que la consulta, tenga un nombre y se pueda referenciar como si de una tabla se tratase.

Las vistas permiten encapsular los detalles de la estructura de tus tablas, pudiendo incluso devolver los datos de forma diferente a cómo lo haría una consulta directa a las tablas. Todo ello, sin perder los cambios que se puedan ir realizando al esquema de base de datos, lo que permite, evolucionar las aplicaciones sin un gran impacto en los interfaces de bases de datos.

Escenarios de uso:

- **Simplificación de Consultas Complejas:** encapsula JOINS o UNIONS de múltiples tablas, subconsultas o filtros lógicos complejos para que los analistas de datos consuman información sin conocer el modelo interno.
- **Seguridad y Control de Acceso:** oculta columnas sensibles (como datos de pasaportes o totales de cobros en vuelos) y permite otorgar permisos exclusivos SELECT sobre la vista y no sobre las tablas base.
- **Abstracción Lógica y Estabilidad:** actúa como un contrato o interfaz estable. Si una tabla física cambia de nombre o se divide (normalización), la vista se puede redefinir para mantener la compatibilidad con aplicaciones heredadas.

Composición y Parámetros Clave de CREATE TABLE

La creación de una tabla física en PostgreSQL requiere especificar de manera detallada el nombre de la tabla, sus columnas, los tipos de datos apropiados y restricciones que garanticen la consistencia de los datos. Para crear una tabla, se debe usar de forma adecuada el denominado comando CREATE TABLE.

Además de los tipos de datos (como varchar, integer, timestamp o point), PostgreSQL provee mecanismos rigurosos para garantizar la integridad de los datos mediante restricciones (constraints). Las más comunes y utilizadas son:

Constrain	Detalle
NOT NULL	Obliga a que la columna tenga un valor válido, previniendo la introducción de valores nulos o desconocidos
UNIQUE	Asegura que todos los valores en la columna o conjunto de columnas sean únicos en toda la tabla.
PRIMARY KEY	Identificador único y no nulo de cada registro. Crea automáticamente un índice B-Tree sobre la columna.
CHECK	Evalúa una condición lógica booleana sobre los datos antes de permitir la inserción o actualización de la fila.
FOREIGN KEY (REFERENCES)	Vincula registros de una tabla hija con una tabla maestra (referenciada). En PostgreSQL, es de vital importancia configurar las reglas activas de integridad referencial como ON UPDATE CASCADE (para propagar cambios de códigos principales a las tablas hijas) y ON DELETE CASCADE (para eliminar registros hijos huérfanos).

Ejemplos prácticos sobre demodb: CREATE TABLE

Para ampliar el esquema relacional de la base de datos en el esquema bookings, crearemos dos nuevas tablas completamente relacionadas con la información existente del sistema de vuelos.

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

```sql
-- 1. Creación de la Tabla de Recogida de Equipajes (Alias: bc)
CREATE TABLE bookings.baggage_claims (
  claim_id serial PRIMARY KEY, -- Identificador único de reclamo de equipaje
  flight_id integer NOT NULL REFERENCES bookings.flights(flight_id) ON UPDATE
CASCADE ON DELETE CASCADE,
  carousel_no varchar(10) NOT NULL, -- Cinta asignada (ej: 'C1', 'C2')
  status varchar(30) DEFAULT 'Scheduled'::character varying,
  CONSTRAINT chk_claim_status CHECK (
    status IN ('Scheduled', 'Arriving', 'Delivered', 'Completed')
  )
);

-- 2. Creación de la Tabla de Puertas de Embarque (Alias: bg)
CREATE TABLE bookings.boarding_gates (
  gate_no varchar(10) NOT NULL, -- Código de pta de embarque (ej: 'A12', 'B05')
  airport_code bpchar(3) NOT NULL REFERENCES
bookings.airports_data(airport_code) ON UPDATE CASCADE ON DELETE CASCADE,
  terminal varchar(10)NOT NULL, -- Terminal del aeropuerto (ej: 'T4', 'T1')
  PRIMARY KEY (gate_no, airport_code) -- Clave primaria compuesta por puerta y
aeropuerto
);

/*
Para simular una operación real dentro del esquema, poblaremos cada una de las
nuevas tablas con un conjunto de 5 registros válidos, respetando las claves
foráneas hacia los aeropuertos ('MAD', 'BCN', 'VLC', 'JFK', 'LHR') y los
identificadores de vuelos registrados en el sistema:
*/

-- Insertar registros en bookings.baggage_claims (Asumiendo IDs de vuelos
existentes en flights)
INSERT INTO bookings.baggage_claims (flight_id, carousel_no, status) VALUES
(1234, 'C1', 'Scheduled'),
(5678, 'C2', 'Arriving'),
(9012, 'C3', 'Delivered'),
(3456, 'C1', 'Completed'),
(7890, 'C2', 'Scheduled');

-- Insertar registros en bookings.boarding_gates (Asumiendo códigos de
aeropuertos válidos)
INSERT INTO bookings.boarding_gates (gate_no, airport_code, terminal) VALUES
('A12', 'MAD', 'T4'),
('B05', 'BCN', 'T1'),
('C02', 'VLC', 'T1'),
('A01', 'JFK', 'T4'),
('B10', 'LHR', 'T2');

-- Comprobamos la creación de la tabla bookings.baggage_claims
SELECT *
FROM    bookings.baggage_claims;
-- Comprobamos la creación de la tabla bookings.boarding_gates
```

PONTIA

```
SELECT *
FROM    bookings.boarding_gates;
```

Ejemplos prácticos sobre demodb: CREATE VIEW

A continuación, presentamos los scripts para generar 3 vistas de base de datos. La primera se basa en las tablas maestros del esquema base (flights y routes), y las siguientes 2 vistas están diseñadas para interactuar directamente con nuestras dos nuevas tablas.

```
-- 1. Vista de Itinerario de Vuelos (Alias: v_fs)
CREATE VIEW bookings.vw_flight_schedules
AS
SELECT fl.flight_id,
       rt.route_no,
       fl.scheduled_departure,
       fl.scheduled_arrival,
       fl.status
FROM    bookings.flights fl
  INNER JOIN bookings.routes rt ON fl.route_no = rt.route_no
WHERE   fl.status = 'Scheduled';
-- Verificamos la creación de la vista
SELECT v_fs.*
FROM    bookings.vw_flight_schedules v_fs;

-- 2. Vista de Recogidas de Equipaje Activas (Alias: v_ac)
CREATE VIEW bookings.vw_active_claims
AS
SELECT bc.claim_id, bc.carousel_no, fl.flight_id, rt.route_no, bc.status
FROM    bookings.baggage_claims bc
  INNER JOIN bookings.flights fl ON bc.flight_id = fl.flight_id
  INNER JOIN bookings.routes rt ON fl.route_no = rt.route_no
WHERE   bc.status != 'Completed';
-- Verificamos la creación de la vista
SELECT v_ac.*
FROM    bookings.vw_active_claims v_ac;

-- 3. Vista de Puertas de Embarque por Aeropuerto y Ciudad (Alias: v_ag)
CREATE VIEW bookings.vw_airport_gates
AS
SELECT bg.gate_no, bg.terminal, ai.airport_code, ai.airport_name, ai.city,
ai.country
FROM    bookings.boarding_gates bg
  INNER JOIN bookings.airports ai ON bg.airport_code = ai.airport_code;
-- Verificamos la creación de la vista
SELECT v_ag.*
FROM    bookings.vw_airport_gates v_ag;
```

ALTER TABLE

La modificación de la estructura de las tablas que ya contienen datos es un escenario recurrente e inevitable en la administración de bases de datos. En PostgreSQL, esto se realiza de manera segura utilizando la familia de comandos ALTER TABLE.

Cuando se crea una tabla y se ha cometido un error, o los requerimientos de la aplicación cambian, se puede eliminar la tabla y crearla de nuevo. Sin embargo, no es la mejor opción si la tabla ya está llena de datos, o si la tabla es referenciada por otros objetos de la base de datos. Por ello PostgreSQL provee una familia de comandos para realizar modificaciones a las tablas existentes.

Usos comunes del comando ALTER TABLE

Operación ALTER	Escenario Real / Caso de Uso
ADD COLUMN	Añadir columnas de auditoría (ej: fecha_modificación, usuario) a tablas en producción.
DROP COLUMN	Asegura que todos los valores en la columna o conjunto de columnas sean únicos en toda la tabla.
ALTER TYPE	Modificar el tipo de dato de una columna (ej: cambiar de un código varchar(5) a varchar(10)).
ADD CONSTRAINT	Establecer nuevas reglas críticas del negocio, como agregar una CHECK o una FOREIGN KEY tardía.
RENAME TO / RENAME COLUMN	Ajustar la semántica y los nombres del esquema sin reconstruir físicamente la tabla.

Ejemplos prácticos sobre demodb

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

```
-- Ejemplo 1: Añadir una columna de auditoría de tiempo a la tabla
baggage_claims
-- Verificamos la estructura antes de la modificación
SELECT * FROM bookings.baggage_claims;
-- Realizamos la modificación para añadir la columna last_updated
ALTER TABLE bookings.baggage_claims
ADD COLUMN last_updated timestamp DEFAULT CURRENT_TIMESTAMP;
-- Verificamos la estructura después de la modificación
SELECT * FROM bookings.baggage_claims;

-- Ejemplo 2: Añadir una CHECK constraint a la tabla boarding_gates para evitar
terminales vacías
-- Verificamos que podemos hacer una inserción sin terminal en antes de la
modificación
INSERT INTO bookings.boarding_gates (gate_no, airport_code, terminal)
VALUES ('C03', 'MAD', '');
-- Verificamos la inserción después de la modificación
SELECT * FROM bookings.boarding_gates;
-- Borramos el registro insertado para mantener la integridad de los datos
DELETE FROM bookings.boarding_gates WHERE terminal = '';
-- Realizamos la modificación para añadir la CHECK constraint
ALTER TABLE bookings.boarding_gates
ADD CONSTRAINT chk_terminal_not_empty CHECK (terminal <> '');
-- Verificamos que ya no podemos hacer un insert en la tabla después de la
modificación
INSERT INTO bookings.boarding_gates (gate_no, airport_code, terminal)
VALUES ('C03', 'MAD', '');
-- Verificamos la inserción después de la modificación
SELECT * FROM bookings.boarding_gates;
```

TRUNCATE TABLE

Es una instrucción DDL optimizada para vaciar de forma rápida y completa una tabla o conjunto de tablas. A diferencia de un DELETE sin calificación, TRUNCATE libera directamente los bloques de disco asignados a la tabla.

Requiere de forma predeterminada un bloqueo de tipo ACCESS EXCLUSIVE y omite la ejecución de disparadores (TRIGGERS) de tipo BEFORE/AFTER DELETE

DROP TABLE

Elimina de forma física y permanente la tabla, su estructura, datos, índices, disparadores y restricciones asociadas en el sistema. Si existen otros objetos que dependen lógicamente de la tabla (como vistas, claves foráneas, etc.), PostgreSQL bloqueará el comando por seguridad a menos que se aplique la cláusula CASCADE, la cual se encarga de eliminar recursivamente todos los objetos dependientes.

DROP evalúa y borra registro por registro de manera incremental escribiendo copioso volumen en el WAL y requiriendo un VACUUM posterior para recuperar el espacio en disco.

Ejemplos prácticos sobre demodb para DROP y TRUNCATE

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

```sql
-- 1. Creamos tablas temporales para almacenar información de vuelos retrasados
o cancelados y reservas de alto valor (VIP)
-- Crear tabla temporal de resumen de vuelos retrasados o cancelados
CREATE TEMPORARY TABLE temp_flights_summary AS
SELECT fl.flight_id,
       fl.route_no,
       fl.status,
       fl.scheduled_departure
FROM bookings.flights fl
  WHERE fl.status IN ('Delayed', 'Cancelled')
LIMIT 1000;
-- Crear tabla temporal de reservas de alto valor (VIP)
CREATE TEMPORARY TABLE temp_bookings_summary AS
SELECT bk.book_ref,
       bk.book_date,
       bk.total_amount
FROM bookings.bookings bk
  WHERE bk.total_amount > 50000.00
LIMIT 1000;
-- Verificamos que las tablas temporales se han creado correctamente
SELECT * FROM temp_flights_summary;
SELECT * FROM temp_bookings_summary;

-- 2. Vaciar rápidamente la tabla temporal de vuelos utilizando TRUNCATE
TRUNCATE TABLE temp_flights_summary;
-- Verificamos que la tabla temporal de vuelos se encuentra vacía
SELECT COUNT(*) FROM temp_flights_summary; -- Retornará 0 registros

-- 3. Eliminar de forma permanente ambas tablas de la sesión temporal (DROP)
DROP TABLE temp_flights_summary;
DROP TABLE temp_bookings_summary;
-- Verificamos que las tablas temporales se han eliminado correctamente
SELECT * FROM temp_flights_summary;
SELECT * FROM temp_bookings_summary;
```

PontIA

COMMENT

Esta sentencia permite enriquecer el catálogo de PostgreSQL almacenando explicaciones técnicas directamente en el diccionario de datos de la base de datos (dentro de las tablas del sistema como pg_description[2]).

Esto es crucial para el mantenimiento de arquitecturas de datos empresariales de gran escala, ya que los comentarios pueden ser extraídos automáticamente por herramientas de documentación o mostrados en clientes SQL.

[2] Es una tabla del sistema en PostgreSQL que guarda comentarios y descripciones de los objetos de la base de datos. Sus funciones principales son almacenar anotaciones creadas con la orden COMMENT, organizar metadatos y ayudar a documentar tablas o columnas

Cuándo utilizar la sentencia COMMENT

- **Autodocumentación:** almacenar el significado de códigos técnicos o acrónimos en columnas específicas.
- **Integración de Catálogos:** para permitir que sistemas externos (herramientas de Gobernanza de Datos o Data Catalogs) recolectan la descripción de las entidades de forma centralizada sin mantener manuales PDF desactualizados.

- **Facilidad en Entornos IT:** como ayuda a los nuevos desarrolladores de software a comprender el esquema relacional directamente desde su consola de administración ejecutando sentencias de introspección.

Ejemplos prácticos sobre demodb:

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

```
-- Ejemplo 1: Añadir un comentario descriptivo a la tabla de boarding_gates
COMMENT ON TABLE bookings.boarding_gates
IS 'Tabla que registra las puertas de embarque físicas de cada aeropuerto.';
-- Verificamos que el comentario se ha añadido correctamente
SELECT obj_description('bookings.boarding_gates'::regclass);

-- Ejemplo 2: Añadir un comentario a una columna de la tabla baggage_claims
COMMENT ON COLUMN bookings.baggage_claims.carousel_no
IS 'Número de cinta transportadora asignada para la recogida de equipajes por
vuelo.';
-- Verificamos el comentario se ha añadido correctamente
SELECT col_description('bookings.baggage_claims'::regclass,
'carousel_no'::regclass);

-- Consultamos los comentarios de las tablas y columnas de la base de datos
SELECT *
FROM    pg_description
WHERE   objoid = 'bookings.boarding_gates'::regclass
   OR   objoid = 'bookings.baggage_claims'::regclass;
```

Anexos

Conceptos clave del comportamiento descrito que aplica a DELETE

Este comando se ejecuta registro por registro, procesando cada fila evaluando condiciones lógicas. Al realizar esta acción, escribe un registro de cambio masivo en el disco. Esto lo hace en el volumen WAL (Write-Ahead Log). De esta forma, se asegura la recuperación ante fallos.

Es por ello que, si el conjunto eliminado de datos es muy grande, puede ser necesario el uso de VACUUM, dado que esta operación deja espacios vacíos ("tuplas muertas") que no liberan el espacio al sistema operativo de inmediato (en el volumen de disco dónde se encuentre WAL), requiriendo limpieza posterior.

```
- VACUUM con ANALYZE (Recomendado)
VACUUM ANALYZE nombre_tabla;
- Opción agresiva (Recuperar espacio en disco)
VACUUM FULL nombre_tabla;
```